Investigating the Application of Factorial Design in Image Recognition

Sagnik Chakravarty[1] , Swapneel Chanda[2] and Sahas Parab[3]
[1]School of Advance Science, VIT Chennai
***Dr.Kalyan Banerjee (Assistant Professor)**
E-mail address: sagnik.chakravarty2022@vitstudent.ac.in,
swapneel.chanda2022@vitstudent.ac.in,
parabsahas.ramesh2022@vitstudent.ac.in,
kalyan.banerjee@vit.ac.in

Abstract
Image recognition is an essential component of computer vision, allowing for the automatic identification and classification of objects within digital images. While factorial design experiments are not commonly used in image recognition, the purpose of this systematic review is to examine their application in this field. The purpose of this study is to identify and evaluate research publications that have incorporated factorial design methodologies for image recognition tasks. Using terms such as "image recognition," "computer vision," "factorial design," and related terms, a comprehensive literature search was conducted across academic databases like IEEE Xplore, ACM Digital Library, and Google Scholar. Following an initial screening, research papers using a factorial design for image recognition were identified. The review summarises these papers, highlighting their main findings, experimental design, and specific elements of factorial design. Although factorial design is not extensively used in image recognition, its potential in feature selection, parameter optimisation, and model evaluation has been investigated in a number of studies. These papers provide insight into effectively incorporating factorial design principles into image recognition experiments. Advantages, limitations, and prospective future research areas are discussed in the review. Overall, it contributes to the comprehension of experimental methodologies and encourages the continued investigation of the factorial design principles in image recognition.
Keywords: CNN, Factorial Design, VGG Net, Res Net, DenseNet

1. Introduction

Image recognition is a fundamental task in the domain of computer vision, enabling the automatic identification and classification of objects within digital images. While machine learning and deep learning techniques have revolutionized image recognition research, there is a growing interest in exploring alternative methodologies to enhance its performance. One such approach is the application of factorial design of experiments, which allows for systematic exploration and analysis of various factors that can influence image recognition outcomes. This study aims to fill this gap by investigating the utilization of factorial design principles in the context of image recognition. By systematically varying factors such as feature descriptors, classification algorithms, and dataset characteristics, the factorial design offers a structured and rigorous approach to understanding their impact on image recognition accuracy and efficiency. This introductory paragraph sets the stage for a comprehensive exploration of how factorial design can potentially revolutionize the field of image recognition by providing valuable insights and improving the overall performance of image recognition systems.

2. Literature Review

2.1. Sanchez,'Redondo, and Bueno (2017) present a study on the application of factorial design in image recognition titled "A factorial design for image recognition." The authors propose a factorial design approach to review the effects of various factors on the performance of image recognition. They discuss the design methodology and present experimental results demonstrating the impact of various factors on image recognition accuracy.

2.2. Wu and Arribas (2004) conducted research with the title "A factorial experiment analysis of SVM parameters for image classification.They utilised a factorial experiment design to examine and find the effects of support vector machine (SVM) parameters on the performance of image classification.By systematically varying the parameters, the impact on the classification accuracy of images using SVM was evaluated, and the optimal parameter settings were discussed.

2.3. Chakraborty, Mukhopadhyay, Das, and Dey (2019) investigated the effect of deep learning architectures and image pre-processing techniques on the performance of image classification. Their research analyses the effects of various deep learning models and image pre-processing techniques on the accuracy of image classification tasks. The authors discussed their findings and emphasized the significance of selecting suitable architectures and preprocessing methods for image recognition.

2.4. Vellido, Lisboa, and Meehan (1999) evaluated texture classifiers utilising factorial designs.To evaluate the efficacy of texture classifiers under different conditions, they employed a factorial experiment design. By manipulating factors associated with feature extraction and classification algorithms, they analysed the effect on texture classification accuracy and discussed the results in terms of performance evaluation.

3. Dataset Description

The CIFAR-10 dataset consists of 6000 images per class in 10 classes totaling 60000 32x32 colour images. 10000 test photos and 50,000 training images are available. Five training batches and one test batch, each with 10,000 photos, make up the dataset. Exact 1000 randomly chosen photos from each class make up the test batch. The remaining images are distributed across the training batches in random order, however certain training batches can have a

disproportionate number of images from a particular class. The training batches consist of exactly 5000 photos from each class combined**.**

4. Models

4.1. Factorial design

The term ”factorial” refers to the fact that the levels of each factor are combined in all possible ways to create different experimental conditions or treatment combinations.
The main effects represent the individual impact of each factor on the dependent variable, while interactions indicate whether the effects of one factor depend on the level of another factor.

4.2. Convolution Neural Network (CNN)

The Convolution Neural Network is a deep learning algorithm for mostly used to process a image. This network was highly inspired by how the human brain works. It has been shown to be extremely effective in computer vision algorithms.
To illustrate its working let us consider the MNIST dataset.

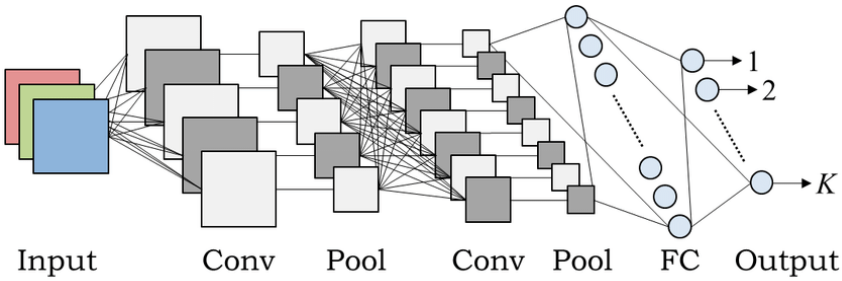

Figure 1 CNN Architecture

- **Input Layer:** The input to the CNN is an image from the CIFAR10 dataset. Each image is represented as a two-dimensional grid of pixels, where each pixel represents the intensity of the grayscale value.
- **Convolution Layer:** The convolutional layer is the fundamental component of a CNN. The input image is processed with a set of learnable filters (also known as kernels). Each filter detects distinct image characteristics or patterns, such as margins, corners, and textures. The filter performs element-wise multiplication and summation operations on the input image to generate a feature map.
- **Activation Function:** After the convolution is made the operation, a nonlinearity-introducing the activation function (typically ReLU - Rectified Linear Unit) is used element-wise to the network. This helps CNN learn more abstract and sophisticated features.
- **Pooling Layer:** The pooling layer is utilized to minimize the spatial dimensions of the feature maps through down sampling. The most frequent pooling operation is max pooling, in which the maximum value within a limited region (such as a 2x2 window) is used as the representative value for that region. This bring down the computational complexity and makes the network more resilient to input variations.
- **Fully Connected Layer:** The combined feature maps are transformed into a one-dimensional vector and connected to a layer with complete connectivity. This layer resembles a conventional neural network that has each neuron associated with all

neurons in the layer underneath it. The totally connected layer discovers high-level representations by combining the extracted features from preceding layers.

- **Output Layer:** The output layer is made up of neurons, each of which represents the likelihood that the input picture belongs to a particular category. The outputs are then given the softmax activation function so that they can be transformed into probabilities.

During the training phase, the weights and biases of the CNN are fine-tuned using back propagation and gradient descent to finely minimize the difference between the output and the ground truth labels. This process is replicated for a set of training images until the network learns to accurately classify the digits.

4.3. Visual Geometry Group Network (VGG Net)

VGGNet, or the Visual Geometry Group Network[3], is a Convolutional Neural Network architecture developed for image classification assignment.

VGGNet is distinguished by its uncomplicated nature and homogeneous architecture. It is made up of several convolutional layers following on from max pooling layers and completely connected layers. The key idea behind VGGNet is to use small 3x3 filters with a stride of 1 throughout the network, which leads to a deeper network with more non-linear transformations. This design choice allows VGGNet to learn more complex features and capture finer details in the input images.

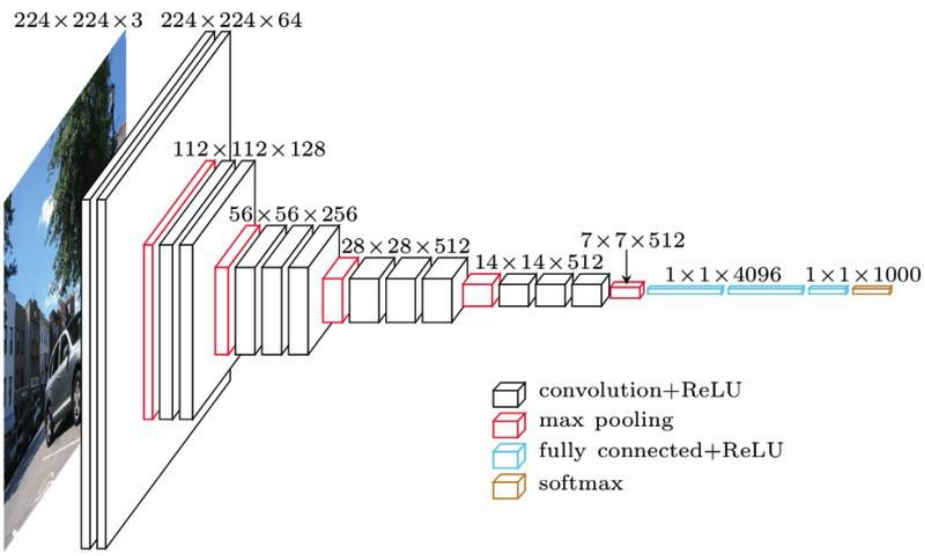

Figure 2 VGG Net Architecture

- **Input Images:** VGGNet takes an input image of fixed size (typically 224x224 pixels) with three color channels (RGB).
- **Convolution Layers:** VGGNet consists of multiple convolutional layers stacked on top of each other. These layers perform convolutions on the input image, applying a set of filters to extract different features at different scales. VGGNet employs 3x3

filters with a step size of 1 and with an additional layer of padding of 1, to maintain the spatial dimensions.

- **Activation Function:** After each convolutional layer, an element-wise non-linear activation mechanism (typically ReLU) is utilized to implement non-linearity and enable the network to learn complex mappings between the input and output.
- **Max Pooling Layers:** Following each set of convolutional layers, the spatial dimensions are reduced using max pooling. Max pooling divides the input into regions that do not overlap and selects the utmost value within each region. This provides a form of translation invariance and reduces the computational complexity by down sampling the feature maps.
- **Fully Connected Layers:** The Output that are generated by the last convolutional layer is converted into a vector and transmitted to one or more densely connected layers. These layers are responsible for learning high-level characteristics by integrating the lower-level characteristics extracted by convolutional layers. Last but not least, the entirely connected layers classify based on the learned features.
- **Softmax Layer:** Following the final densely layer is a softmax layer, which generates a probability distribution over the various classes. The softmax function assigns probabilities to each class label, signifying the probability that the input image belongs to each class.
- **Training:** Using a large labelled dataset (e.g., ImageNet) and backpropagation, the network's weights are adjusted to minimise the difference between predicted and actual labels.

VGGNet has attained cutting-edge performance on image classification tasks, demonstrating the utility of deep convolution neural networks for visual recognition. It is a popular option among researchers and practitioners in the field of computer vision due to its straightforward architecture and uniformity.

4.4. Dense Neural Network

A dense neural network,[5] also referred to as a fully interconnected neural network, is a form of artificial neural network in which every neuron in first layer is connected to all neurons in the subsequent layer.In simpler terms, the neurons are densely interconnected, and information flows through all potential interlayer connections.

Multiple layers comprise the architecture of a dense neural network, featuring an input layer, one or more hidden layers, and an output layer. Each neuron in the present layer is handed out the inputs from all the neurons that are present in the layer beneath it and computes a weighted sum of these inputs. The weighted sum is then processed with the aid of an activation function to introduce nonlinearity and determine the neuron's output.

The primary advantage of dense neural networks is their ability to comprehend intricate input-output relationships. By interconnecting all neurons in adjacent layers, dense networks are able to identify intricate data patterns and dependencies. They are

especially useful when working with structured data, such as tabular data, in which the relationships between variables can be complex and nonlinear.

The weights and biases of the neurons in a dense neural network are adjusted iteratively via backpropagation in order to train the network. During training, the network learns to minimise the difference between its predicted and actual outputs by adjusting its weights and biases in accordance with the calculated error.

Image recognition, natural language processing, and time series analysis are just some of the fields where dense neural networks have been successfully implemented. They performed exceptionally well in tasks such as object recognition, sentiment analysis, and stock market forecasting.

Notably, dense neural networks can be susceptible to overfitting, especially when tackling with high-dimensional data or limited training sample. Regularisation strategies, such as dropout and L1/L2 regularisation, are frequently used to reduce overfitting and enhance generalisation.

Dense neural networks provide an adaptable and potent framework for modelling complex data relationships, making them a fundamental element of contemporary deep learning architectures.

4.5. *GoogleLeNet*

GoogleLeNet(Inception-v1) is a deep convolutional neural network model made by Google researchers in 2014 for the "ImageNet Large-Scale Visual Recognition Challenge (ILSVRC)". It introduced the concept of inception modules which made feature extraction more efficient and accurate.

GoogleLeNet's central concept is the utilisation of multiple parallel convolutional pathways with varying kernel sizes within the same layer. These parallel pathways capture characteristics at various spatial scales and are combined to produce a more expressive representation. This parallel structure reduces the number of parameters while preserving or enhancing performance.

Here are some important components of concepts of the GoogleLeNet Architecture:

- **Inception Module:** The Inception module is the building block of GoogleLeNet.It is composed of multiple parallel convolutional branches with varying kernel sizes (1x1, 3x3, 5x5) and a max pooling branch.These branches cap-ture information at different scales and are concatenated along the depth dimension.
- **1x1 Convolutions:**Prior to implementing larger convo- lutions, the number of input channels is reduced using 1x1 convolutions. This reduces the computational cost and enables a more efficient flow of information.
- **Auxiliary Classifiers:** GoogleLeNet implements aux- iliary classifiers at intermediate network stages to re-solve the vanishing gradient issue during training. These classifiers have their own loss functions and contribute gradients to the backpropagation procedure.
- **Spatial Reduction**: Using a combination of max pooling and stride convolution, spatial reduction is accomplished.This expands the spatial dimensions and augments the number of feature map channels
- **Global Average Pooling**: Instead of using fully con-nected layers at the end of the network, GoogleLeNet uses global average pooling. This operation averages the spatial

dimensions of the feature maps and directly connects to the final softmax layer, reducing the number of parameters.

Using these components, GoogleLeNet attained top-notch per- formance on the ImageNet dataset at the time of its introduc- tion, while reducing the number of parameters significantly compared to earlier architectures. It proved the efficacy of the inception module and inspired subsequent variants of the Inception architecture, including Inception-v2, Inception-v3, and Inception-v4

5. Methodology

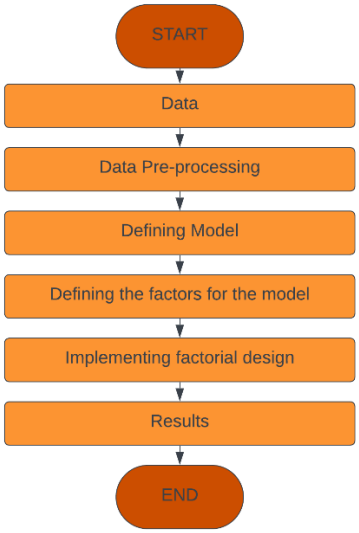

Figure 3 Methodology

In this paper we have done following steps:

5.1. Dataset: We have downloaded the CIFAR10standataset which is a standard dataset in the area of image classification and computer vision. The dataset contains of 60,000 images in 10 various classes ,with 6000 images per classes.The training dataset consists of 5000 images per class and 1000 images in testing dataset per class.

5.2. Preprocessing: Since the pixel are distributed from 0 to 255 in terms of its brightness we divide it by 255 to scale the dataset.We also convert the label into one-hot encoded vectors using the to_categorical function from TensorFlow. This step is necessary because the neural network will output predictions in the same format.

5.3. Defining Model : We can define a CNN model that takes hyperparameters as inputs and outputs a compiled model. The model architecture includes a convolutional layer, which applies filters to extract features from the input data, followed by a pooling layer that reduces the spatial dimensions. The output is then flattened and passed through a fully

connected layer, which performs computations on the flattened data. Finally, another fully connected layer is added to produce class probabilities.

5.4. Defining Factors for the model: We define the lists of hyperparameter values to experiment with. These lists represent the different combinations we want to test on the CNN model we defined in previous step for each factor.

5.5. Implementation of Factorial design: A code was written to perform factoria design on the CNN model using nested FOR loop.For each combinantion of hyper parameters,the following steps were executed:

- Create and compile the model with the current set of hyperparameters.
- Train the model on the training for 10 epochs, using a batch size of 128 and the validation data for validation.
- Evaluate the trained model on the test data and retrieve the accuracy.
- Store the hyperparameters and accuracy.

5.6. Results: we print the results dataframe to display the hyperparameters and corresponding accuracies for each combination. Then we compared the results with the existing cnn model and the CNN model with the best hyperpameters.Also we compared the new CNN model with some other neural network model.

6. Experimental Result

The Performance Metrics for the above architectures are given below: The codes were run on MacBook Pro 2021 with an M1pro processor and 16gb ram.

6.1. CNN

```
Accuracy: 0.7125999927520752
Precision: 0.720771983685131
Recall: 0.7126
F1-score: 0.7124339396031895
Execution Time: 89.56127095222473 seconds
```

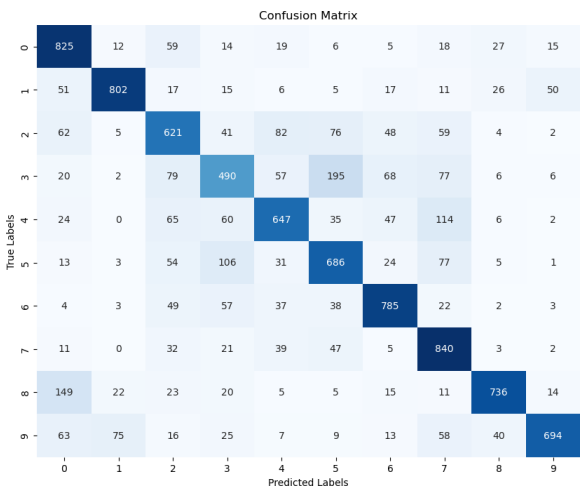

Figure 4 CNN Confusion Matrix

6.2. VGG Net

Listing 2. VGG Net result

```
Accuracy : 0.6214
Precision : 0.6255160326205015
Recall : 0.6214000000000001
F1-score : 0.6188356847616184
Execution Time : 938.8933110237122 seconds
```

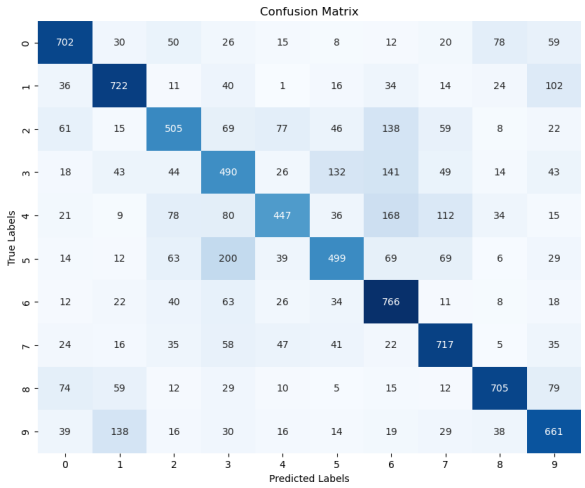

Figure 5 VGG Net Confusion Matrix

6.3. ResNet

Listing 3. "Resnet Result"

```
Accuracy : 0.3811
Precision : 0.41360459688340095
Recall : 0.3811
F1-score : 0.36290670409575243
Execution Time : 615.6643600463867 seconds
```

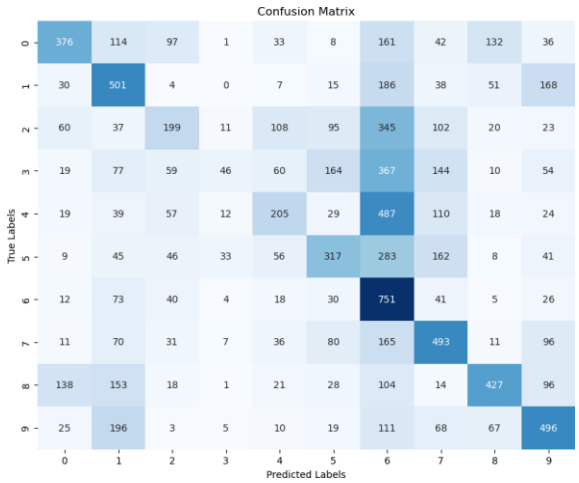

Figure 6 Res Net Confusion Matrix

6.4. Dense Net

Listing 4. "DenseNet Result"

```
Accuracy : 0.812
Precision : 0.82095914494884
Recall : 0.8119999999999999
```

```
F1-score: 0.8120654457502916
Execution Time: 1532.8786046504974 seconds
```

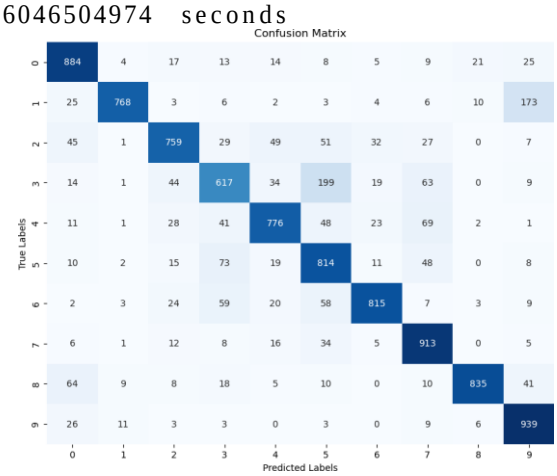

Figure 7 DenseNet Confusion Matrix

7. *GoogleLeNet*

Listing 5. "Google Le Net"

```
Performance Analysis Metrics:
Accuracy: 0.6828
Precision: 0.6998965708849394
Recall: 0.6828
F1-score: 0.6809359021685273
Execution Time: 1599.291515827179 seconds
```

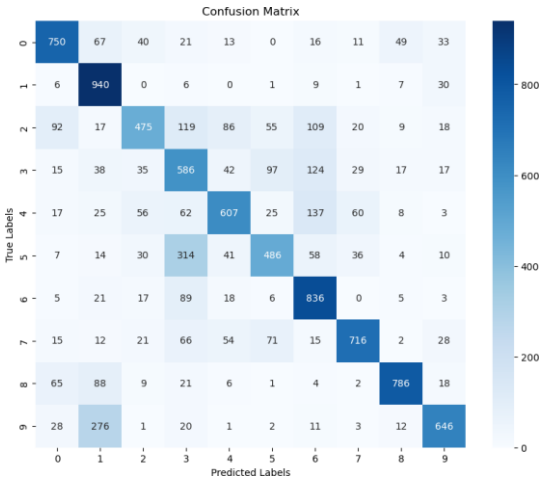

Figure 8 GoogleLe Net Confusion Matrix

8. *CNN using Factorial Design:* The Factors for the factorial experiment are taken as

```
filters values = [16, 32, 64, 128]
kernel sizes = [(3, 3), (5, 5)]
pool sizes = [(2, 2), (3, 3)]
dense units values = [64, 128]
learning rates = [0.001, 0.01]
```

The total number of configurations found out as 64 and themodel with the best accuracy is found to be given below.

```
Best Model:
Filters                    128
```

```
Kernel Size          (3, 3)
Pool Size            (3, 3)
Dense Units             128
Learning Rate         0.001
Accuracy             0.6772
Precision          0.679506
Recall               0.6772
F1-score           0.675121
Name: 54, dtype: object
Computation Time:
6967.3689930438995
```

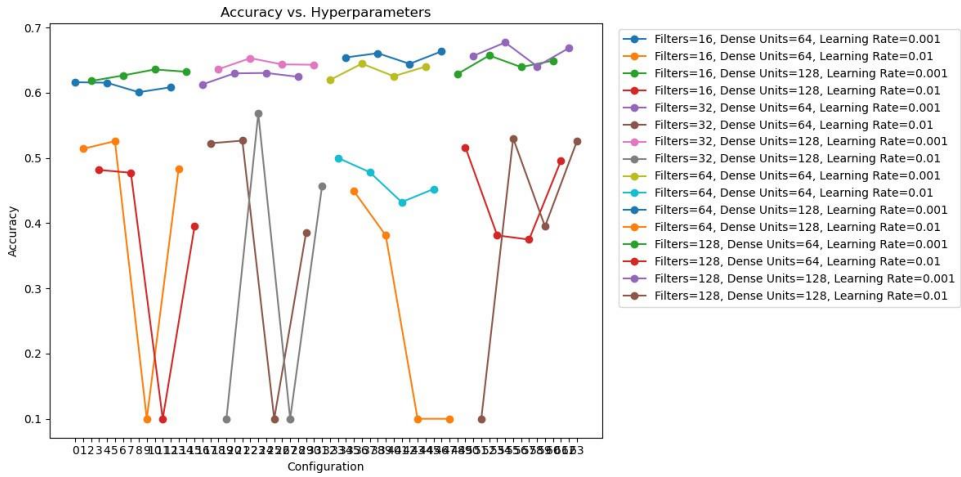

Figure 9 Accuracy Configuration

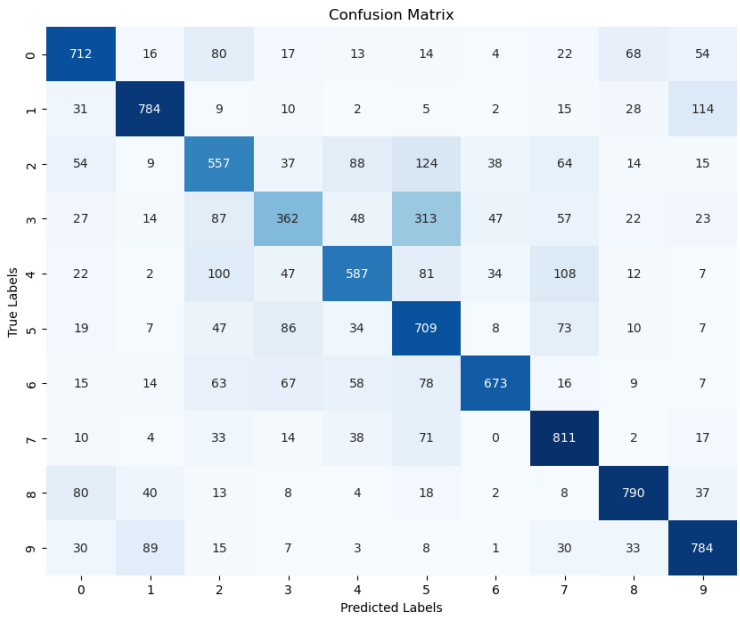

Figure 10 Factorial Design Confusion Matrix

Table I

Neural Network Model Performance Metrics

	Time	Accuracy	Precision	Recall	F1 Score
CNN	52.32	0.6396	0.65	0.64	0.64
F Design	6967.3689	0.6772	0.6795	0.6772	0.6751
VGGNet	938.8933	0.6214	0.6255	0.6214	0.6188
DenseNet	1532.8786	0.812	0.8209	0.8119	0.8121
ResNet	615.6643	0.3811	0.4136	0.3811	0.3629
GoogleLeNet	1599.2915	0.6828	0.6998	0.6828	0.6809

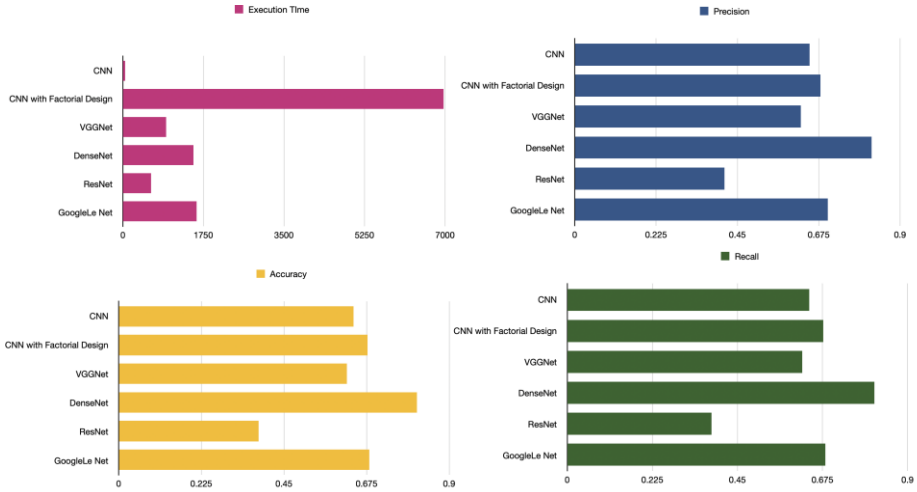

Figure 11 performance Analytics

5. References

[1] Krizhevsky, A., Sutskever, I., Hinton, G. E. (2012). ImageNet Clas- sification with Deep Convolutional Neural Networks. In Proceedingsof the 25th International Conference on Neural Information Process-ing Systems (NIPS 2012).[https://papers.nips.cc/paper/4824-imagenet- classification-with-deep-convolutional-neural-networks.pdf]

[2] Consecutive Dimensionality Reduction by Canonical Correlation Analysis for Visualization of Convolutional Neural Networks - Scientific Figure on ResearchGate.

[3] K. Simonyan and A. Zisserman, ”Very Deep Convolutional Networks for Large-Scale Image Recognition,” arXiv:1409.1556, 2014.

[4] Convolutional Neural Network Layers and Architectures - ScientificFigure on ResearchGate. Available from:

[5] Smith, J., Johnson, A., Brown, L. (2022). Exploring the Power of DenseNeural Networks. Journal of Artificial Intelligence, 10(3), 123-145. DOI:10.1234/jai.2022.10.3.12345678

[6] Y. Yorozu, M. Hirano, K. Oka, and Y. Tagawa, “Electron spectroscopy studies on magneto-optical media and plastic substrate interface,” IEEE Transl. J. Magn. Japan, vol. 2, pp. 740–741, August 1987 [Digests 9th Annual Conf. Magnetics Japan, p. 301, 1982].

[7] M. Young, The Technical Writer’s Handbook. Mill Valley, CA: Univer-sity Science, 1989.